EventHub Project Documentation (Section-wise)
Generated on: 2026-03-27 00:23:10

1. Project Overview
EventHub is a full-stack event platform with role-based access:
- Public pages: home, register, login.
- User dashboard: browse events, register, tickets, bookings, calendar, profile, support.
- Organizer dashboard: organizer home with managed-event insights.

Core backend is implemented in Python (standard library HTTP server) and stores data in MySQL.
Frontend is plain HTML/CSS/JavaScript and consumes backend JSON APIs using fetch().

2. Technology Stack Used
2.1 Backend
- Language: Python 3
- Server: http.server.BaseHTTPRequestHandler with ThreadingHTTPServer
- Session model: in-memory dict (SESSIONS) + HttpOnly cookie session_id
- Database driver: mysql-connector-python (import mysql.connector)
- Utility: standard lib modules for hashing, cookies, email, URL parsing, MIME serving

2.2 Database
- Database: MySQL
- Schema bootstrap + migration-style column checks performed at server startup (init_db)

2.3 Frontend
- HTML/CSS/Vanilla JS pages under webapp/
- User dashboard logic: webapp/dashboard/user/user.js
- Organizer dashboard logic: webapp/dashboard/organizer/organizer.js
- Communication: fetch() to backend endpoints (JSON + form-urlencoded + multipart)

3. Server-side Programming: How It Is Implemented
3.1 Entry point
- File: app.py
- run() calls init_db(), then starts ThreadingHTTPServer at 127.0.0.1:8000.

3.2 Routing model
GET routes include:
- /home (and /)
- /register
- /login
- /dashboard/user
- /dashboard/organizer
- /userdashboardservlet
- /organizerdashboardservlet
- /ticketsservlet
- /bookingsservlet
- /profileservlet
- /registeredeventsservlet
- /eventscalendarservlet
- /logout
- /assets/... (static files from webapp)

POST routes include:
- /register
- /login
- /register-event
- /update-booking
- /cancel-booking
- /update-profile
- /update-profile-password
- /contact
- /upload-profile-image

3.3 Session and auth flow
- After successful login, a random token is generated via secrets.token_urlsafe(32).
- Token is stored in SESSIONS with user identity and role.
- Cookie session_id is returned with HttpOnly and SameSite=Lax.
- Protected handlers call current_user()/require_session() and enforce role checks.
- Role gating:
  - organizer endpoint/page access requires role == organizer
  - user endpoints require role == user

3.4 Static and template serving
- HTML files are read from webapp/ and served by serve_html/send_html.
- Assets are served from /assets/... using resolve_asset() with path-safety checks.

4. Database Connection and Schema
4.1 Connection configuration
DB_CONFIG reads from env vars with defaults:
- EVENTHUB_DB_HOST (default localhost)
- EVENTHUB_DB_PORT (default 3306)
- EVENTHUB_DB_USER (default root)
- EVENTHUB_DB_PASSWORD
- EVENTHUB_DB_NAME (default eventhub)

4.2 Connection helper
- get_connection() returns mysql.connector.connect(**DB_CONFIG)

4.3 Startup bootstrap and schema creation
init_db() performs:
- CREATE DATABASE IF NOT EXISTS eventhub
- CREATE TABLE IF NOT EXISTS users
- CREATE TABLE IF NOT EXISTS user_event_registrations
- CREATE TABLE IF NOT EXISTS contact_messages

It also runs add_column_if_missing(...) to safely evolve schema for fields like:
- users.phone, users.bio, users.profile_image
- user_event_registrations.attendee_name, attendee_email, attendee_phone, ticket_type, ticket_count, city, payment_method, address, special_request, consent_accepted, booking_status, canceled_at

4.4 Important table notes

- users.email is unique.
- user_event_registrations has UNIQUE KEY (user_email, event_id).
  This is key to how registration works: re-registering same event updates existing row
  via ON DUPLICATE KEY UPDATE instead of creating duplicate rows.

5. Frontend <-> Backend <-> Database Integration
5.1 Main pattern
- Frontend sends fetch requests.
- Backend validates session + payload.
- Backend performs MySQL query.
- Backend returns JSON.
- Frontend updates DOM/UI state.

5.2 Concrete examples
A) Browse registration
- Frontend: POST /register-event with attendee details, ticket count, consent.
- Backend: validates fields and inserts/updates user_event_registrations.
- Backend response includes ticket payload.
- Frontend marks card as Registered and refreshes tickets/bookings/dashboard sections.

B) Tickets page
- Frontend: GET /ticketsservlet
- Backend: derives active tickets from booking rows + event catalog.
- Frontend renders cards and supports download/share.

C) Bookings page
- Frontend: GET /bookingsservlet
- Backend: fetches booking records for logged-in user.
- Frontend supports edit/cancel actions through:
  - POST /update-booking
  - POST /cancel-booking

D) Profile settings
- Frontend: GET /profileservlet then POST /update-profile /update-profile-password
- Backend updates users table and session name/role cache where needed.

E) Profile image upload
- Frontend: multipart POST /upload-profile-image
- Backend parses multipart manually, validates JPG/PNG signature + size,
  stores file under webapp/dashboard/images/profiles, updates users.profile_image.

6. Event Data Model in Current Build
- Event catalog is currently an in-memory constant list (EVENT_CATALOG) in app.py.
- Each event has id, title, date/time, location, type, seat info, price, status, category, image,
organizer phone.
- Bookings/tickets are persisted in MySQL, while master event metadata is from EVENT_CATALOG.

Implication:
- Changing event catalog in code changes available browse/calendar events.
- There is no separate events table yet for organizer-created events.

7. Role-based UX Behavior (Current)
7.1 Before login/register
- User can access public pages and browse-related pages depending on route/state.

7.2 New user after login
- Registered events list is fetched from /registeredeventsservlet.
- If no active registrations, restricted sections are hidden/empty:
  my tickets, my bookings, profile-related restricted nav entries.
- Browse cards default to Register Now state.

7.3 After registration
- Browse shows Registered state and allows reopening flow (Register Again button).
- Ticket quantity updates displayed total price in registration modal.
- Tickets and bookings become visible with live data from backend.

8. Organizer Section Status
- Organizer dashboard page consumes /organizerdashboardservlet.
- Current payload comes from sample_organizer_dashboard(...) mock data in backend.
- This means organizer dashboard is dynamic in UI rendering but not yet fully DB-driven.

9. Email/Support Flow
- Contact form posts to /contact.
- Message is always stored in contact_messages.
- Backend attempts SMTP send via Gmail SSL.
- Delivery status and error are written back into DB (sent/failed + error text).

10. Security and Engineering Notes
10.1 Current strengths
- Route-level auth and role checks are present.
- HttpOnly cookie is used.
- Passwords are hashed with SHA-256 before storage (for new updates).
- Asset path resolution has traversal protection.

10.2 Current limitations / improvements suggested
- Sessions are in-memory only (lost on restart; not multi-instance safe).
- No CSRF protection for form endpoints yet.
- Password hashing is plain SHA-256 without salt/stretching.
  Recommended: bcrypt/argon2.
- DB password default is hardcoded fallback in code; should rely fully on env vars.
- Multipart parsing is handcrafted; using a robust parser/framework would reduce risk.
- No explicit rate-limiting for login/contact routes.

11. End-to-End Request Lifecycle (Example)
Example: user registers for an event
1. User opens browse page and clicks Register.
2. Frontend opens modal, submits form data to /register-event.
3. Backend validates session + required fields + consent + event id.
4. Backend UPSERTs user_event_registrations row.
5. Backend returns JSON success with ticket info.
6. Frontend updates card state, reloads tickets/bookings/dashboard widgets.
7. Subsequent pages fetch fresh data from tickets/bookings endpoints.

12. Files That Matter Most
- Backend core: app.py
- User dashboard logic: webapp/dashboard/user/user.js
- User pages: webapp/dashboard/user/*.html
- Organizer dashboard logic: webapp/dashboard/organizer/organizer.js
- Public pages: webapp/home/*, webapp/login/*, webapp/register/*

13. What Has Been Completed Till Now (Functional Summary)
- Authentication: register/login/logout with session cookie.
- Role-based dashboard entry and endpoint protection.
- User browse registration workflow with backend persistence.
- Tickets page with per-ticket actions (preview/share/download logic in frontend).
- Bookings page with edit and cancel APIs.
- Event calendar API and frontend filtering/search behavior.
- Profile details + password update APIs.
- Profile image upload API with DB-backed image URL.
- Contact/support API with DB persistence and optional SMTP dispatch.
- New user experience improvements for empty booking/ticket states.
- Browse page enhancement: ticket-count based price update + Register Again action.

14. Recommended Next Phase
- Move EVENT_CATALOG into a real events table.
- Build organizer CRUD endpoints for create/update/publish events.
- Replace in-memory session with DB/Redis-based session store.
- Upgrade password hashing to bcrypt/argon2.
- Add CSRF tokens and API rate limits.
- Add automated tests for key flows (auth, registration, bookings, profile upload).

15. Quick Run Notes
- Ensure MySQL is running and credentials are available via env vars.
- Start server: python app.py
- App URL: http://127.0.0.1:8000